

Start coding or generate with AI.

Load Dataset

Subtask:

Load the dataset from the file '[/content/boston.csv](#)' into a pandas DataFrame.

```
import pandas as pd
df = pd.read_csv('/content/boston.csv')
print("First 5 rows of the DataFrame:")
print(df.head())
print("\nDataFrame Info:")
df.info()
```

First 5 rows of the DataFrame:

	CRIM	INDUS	NOX	RM	AGE	DIS	TAX	PT	\
0	0.00632	2.31	0.538	6.575	65.199997	4.0900	296	15.300000	
1	0.02731	7.07	0.469	6.421	78.900002	4.9671	242	17.799999	
2	0.02729	7.07	0.469	7.185	61.099998	4.9671	242	17.799999	
3	0.03237	2.18	0.458	6.998	45.799999	6.0622	222	18.700001	
4	0.06905	2.18	0.458	7.147	54.200001	6.0622	222	18.700001	

	B	MV
0	396.899994	24.000000
1	396.899994	21.600000
2	392.829987	34.700001
3	394.630005	33.400002
4	396.899994	36.200001

DataFrame Info:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 506 entries, 0 to 505
```

```
Data columns (total 10 columns):
```

#	Column	Non-Null Count	Dtype
0	CRIM	506 non-null	float64
1	INDUS	506 non-null	float64
2	NOX	506 non-null	float64
3	RM	506 non-null	float64
4	AGE	506 non-null	float64
5	DIS	506 non-null	float64
6	TAX	506 non-null	int64
7	PT	506 non-null	float64
8	B	506 non-null	float64
9	MV	506 non-null	float64

```
dtypes: float64(9), int64(1)
memory usage: 39.7 KB
```

Missing Value Analysis

Subtask:

Perform an initial analysis to identify and quantify missing values in each column of the dataset. Display the count and percentage of missing values. Visualize the distribution of missing values using a heatmap or bar chart.

Reasoning: First, I will calculate the count and percentage of missing values for each column and display them in a DataFrame for clarity. This addresses instructions 1, 2, and 3.

```
missing_values_count = df.isnull().sum()
missing_values_percentage = (df.isnull().sum() / len(df)) * 100
```

```
missing_values_table = pd.DataFrame({
    'Missing Values Count': missing_values_count,
    'Missing Values Percentage': missing_values_percentage
})
```

```
print("Missing Values Analysis:")
print(missing_values_table[missing_values_table['Missing Values Count'] > 0].sort_values(by='Missing Values Count', ascending=False))
```

```
Missing Values Analysis:
Empty DataFrame
Columns: [Missing Values Count, Missing Values Percentage]
Index: []
```

Given the output from the previous step, `missing_values_table` is an `Empty DataFrame`, which indicates that there are no missing values in the dataset. Therefore, creating a heatmap to visualize missing values would result in an empty or uninformative plot. The initial analysis has confirmed the absence of missing data, so no further action is required for handling them.

▼ Outlier Analysis

Subtask:

Identify outliers in the numerical columns of the dataset using appropriate methods and visualize them. Summarize the findings.

Reasoning: First, I will identify the numerical columns from the DataFrame. Then, for each numerical column, I will calculate the first quartile (Q1), the third quartile (Q3), and the Interquartile Range (IQR), and define the upper and lower bounds for outlier detection using the IQR method.

```
numerical_cols = df.select_dtypes(include=['number']).columns

outlier_info = {}

for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)][col]

    outlier_info[col] = {
        'Q1': Q1,
        'Q3': Q3,
        'IQR': IQR,
        'Lower Bound': lower_bound,
        'Upper Bound': upper_bound,
        'Outlier Count': outliers.count(),
        'Outliers': outliers.tolist()
    }

print("Outlier Analysis Summary (IQR method):\n")
for col, info in outlier_info.items():
    print(f"Column: {col}")
    print(f"  Q1: {info['Q1']:.2f}")
    print(f"  Q3: {info['Q3']:.2f}")
    print(f"  IQR: {info['IQR']:.2f}")
    print(f"  Lower Bound: {info['Lower Bound']:.2f}")
    print(f"  Upper Bound: {info['Upper Bound']:.2f}")
    print(f"  Number of Outliers: {info['Outlier Count']}")
    if info['Outlier Count'] > 0:
        print(f"    Example Outliers (first 5): {info['Outliers'][:5]}")
    print("\n")

Outlier Analysis Summary (IQR method):

Column: CRIM
Q1: 0.08
Q3: 3.68
IQR: 3.60
Lower Bound: -5.31
Upper Bound: 9.07
Number of Outliers: 66
Example Outliers (first 5): [13.52219963, 9.232299805, 11.10809994, 18.49819946, 19.60910034]

Column: INDUS
Q1: 5.19
Q3: 18.10
IQR: 12.91
Lower Bound: -14.18
Upper Bound: 37.47
Number of Outliers: 0

Column: NOX
Q1: 0.45
Q3: 0.62
IQR: 0.18
Lower Bound: 0.19
Upper Bound: 0.89
Number of Outliers: 0
```

```

Column: RM
Q1: 5.89
Q3: 6.62
IQR: 0.74
Lower Bound: 4.78
Upper Bound: 7.73
Number of Outliers: 30
Example Outliers (first 5): [8.069000244, 7.820000172, 7.802000046, 8.375, 7.928999901]

```

```

Column: AGE
Q1: 45.02
Q3: 94.07
IQR: 49.05
Lower Bound: -28.55
Upper Bound: 167.65
Number of Outliers: 0

```

```

Column: DIS
Q1: 2.10
Q3: 5.19
IQR: 3.09
Lower Bound: -2.53
Upper Bound: 9.82
Number of Outliers: 5
Example Outliers (first 5): [10.71030045, 10.71030045, 12.12650013, 10.58570004, 10.58570004]

```

Reasoning: Now that the outlier detection has been performed and summarized in text, I will visualize the outliers using box plots for each numerical column to visually inspect their distribution and presence as per the subtask instructions. This will provide a clear graphical representation of the identified outliers.

```

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
df = pd.read_csv('/content/boston.csv')

specific_cols = ['INDUS', 'DIS', 'MV']

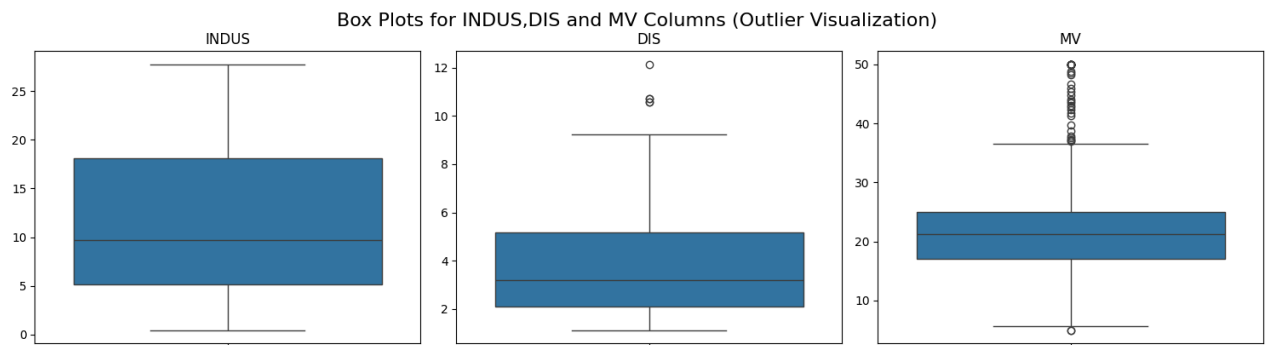
n_cols = 3
n_rows = 1

fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 5, n_rows * 4))
if n_rows == 1 and n_cols > 1:
    axes = axes
elif n_rows > 1 or n_cols > 1:
    axes = axes.flatten()
else:
    axes = [axes]

for i, col in enumerate(specific_cols):
    if col in df.columns:
        sns.boxplot(y=df[col], ax=axes[i])
        axes[i].set_title(col)
        axes[i].set_ylabel('')
    else:
        print(f"Warning: Column '{col}' not found in the DataFrame.")
        fig.delaxes(axes[i])

plt.tight_layout()
plt.suptitle('Box Plots for INDUS, DIS and MV Columns (Outlier Visualization)', y=1.02, fontsize=16)
plt.show()

```



PPA Experiment 4

Start coding or [generate](#) with AI.

PPA EXPERIMENT 4

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Load sample dataset (Titanic dataset from seaborn)
import seaborn as sns
df = sns.load_dataset("titanic")

print("Dataset Preview:")
print(df.head())

# Selecting numerical columns
num_df = df[['age', 'fare']].dropna()

print("\nOriginal Summary:")
print(num_df.describe())

# ----- Fixing Skewness (Log Transformation) -----
num_df['fare_log'] = np.log1p(num_df['fare'])

# ----- Scaling -----
scaler1 = StandardScaler()
num_df['age_standard'] = scaler1.fit_transform(num_df[['age']])

scaler2 = MinMaxScaler()
num_df['fare_minmax'] = scaler2.fit_transform(num_df[['fare']])

# ----- Binning -----
num_df['age_bins'] = pd.cut(num_df['age'], bins=5, labels=["Very Low", "Low", "Medium", "High", "Very High"])

num_df['fare_bins'] = pd.qcut(num_df['fare'], q=4, labels=["Cheap", "Average", "Expensive", "Very Expensive"])

# ----- Sampling -----
sample_data = num_df.sample(n=10, random_state=42)

print("\nProcessed Data Preview:")
print(num_df.head())

print("\nSampled Data (10 rows):")
print(sample_data)

# ----- Visualization -----
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.hist(num_df['fare'], bins=20, edgecolor='black')
plt.title("Original Fare Distribution")

plt.subplot(1,2,2)
plt.hist(num_df['fare_log'], bins=20, edgecolor='black')
```

```
plt.title("Log Transformed Fare Distribution")
```

```
plt.show()
```

Dataset Preview:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

Original Summary:

	age	fare
count	714.000000	714.000000
mean	29.699118	34.694514
std	14.526497	52.918930
min	0.420000	0.000000
25%	20.125000	8.050000
50%	28.000000	15.741700
75%	38.000000	33.375000
max	80.000000	512.329200

Processed Data Preview:

	age	fare	fare_log	age_standard	fare_minmax	age_bins	fare_bins
0	22.0	7.2500	2.110213	-0.530377	0.014151	Low	Cheap
1	38.0	71.2833	4.280593	0.571831	0.139136	Medium	Very Expensive
2	26.0	7.9250	2.188856	-0.254825	0.015469	Low	Cheap
3	35.0	53.1000	3.990834	0.365167	0.103644	Medium	Very Expensive
4	35.0	8.0500	2.202765	0.365167	0.015713	Medium	Cheap

Sampled Data (10 rows):

	age	fare	fare_log	age_standard	fare_minmax	age_bins	\
149	42.0	13.0000	2.639057	0.847383	0.025374	Medium	
407	3.0	18.7500	2.983153	-1.839248	0.036598	Very Low	
53	29.0	26.0000	3.295837	-0.048161	0.050749	Low	
369	24.0	69.3000	4.252772	-0.392601	0.135265	Low	
818	43.0	6.4500	2.008214	0.916271	0.012590	Medium	
549	8.0	36.7500	3.630985	-1.494808	0.071731	Very Low	
85	33.0	15.8500	2.824351	0.227391	0.030937	Medium	
774	54.0	23.0000	3.178054	1.674039	0.044893	High	
100	28.0	7.8958	2.185579	-0.117049	0.015412	Low	
816	23.0	7.9250	2.188856	-0.461489	0.015469	Low	

	fare_bins
149	Average
407	Expensive
53	Expensive
369	Very Expensive
818	Cheap
549	Very Expensive
85	Expensive
774	Expensive
100	Cheap
816	Cheap

PPA EXPERIMENT 6 Original Fare Distribution

Log Transformed Fare Distribution

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Sample Transaction Dataset
transactions = [
    ['Milk', 'Bread', 'Butter'],
    ['Beer', 'Bread'],
    ['Milk', 'Bread', 'Beer', 'Butter'],
    ['Bread', 'Butter'],
    ['Milk', 'Butter'],
    ['Beer', 'Butter']
]

# Convert into One-Hot Encoding
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

print("Transaction Dataset (One Hot Encoded):")
print(df)
```

```
# Apply Apriori Algorithm
frequent_itemsets = apriori(df, min_support=0.3, use_colnames=True)

print("\nFrequent Itemsets:")
print(frequent_itemsets)

# Generate Association Rules
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)

print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

Transaction Dataset (One Hot Encoded):

	Beer	Bread	Butter	Milk
0	False	True	True	True
1	True	True	False	False
2	True	True	True	True
3	False	True	True	False
4	False	False	True	True
5	True	False	True	False

Frequent Itemsets:

	support	itemsets
0	0.500000	(Beer)
1	0.666667	(Bread)
2	0.833333	(Butter)
3	0.500000	(Milk)
4	0.333333	(Bread, Beer)
5	0.333333	(Butter, Beer)
6	0.500000	(Bread, Butter)
7	0.333333	(Milk, Bread)
8	0.500000	(Milk, Butter)
9	0.333333	(Milk, Bread, Butter)

Association Rules:

	antecedents	consequents	support	confidence	lift
0	(Beer)	(Bread)	0.333333	0.666667	1.000000
1	(Beer)	(Butter)	0.333333	0.666667	0.800000
2	(Bread)	(Butter)	0.500000	0.750000	0.900000
3	(Butter)	(Bread)	0.500000	0.600000	0.900000
4	(Milk)	(Bread)	0.333333	0.666667	1.000000
5	(Milk)	(Butter)	0.500000	1.000000	1.200000
6	(Butter)	(Milk)	0.500000	0.600000	1.200000
7	(Milk, Bread)	(Butter)	0.333333	1.000000	1.200000
8	(Milk, Butter)	(Bread)	0.333333	0.666667	1.000000
9	(Bread, Butter)	(Milk)	0.333333	0.666667	1.333333
10	(Milk)	(Bread, Butter)	0.333333	0.666667	1.333333

```
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated
return datetime.datetime.utcnow().replace(tzinfo=utc)
```

Start coding or generate with AI.